

Sharing Knowledge between Independent Grid Communities

Katja Hose¹, Steffen Metzger¹, Ralf Schenkel²

¹Max Planck Institute for Informatics, Saarbrücken, Germany

²Saarland University, Saarbrücken, Germany

Abstract: In recent years, grid-based approaches for processing scientific data became popular in various fields of research. A multitude of communities has emerged that all benefit from the processing and storage power the grid offers to them. So far there has not yet been much collaboration between these independent communities. But applying semantic technologies to create knowledge bases, sharing this knowledge, and providing access to data maintained by a community, allows to exploit a synergy effect that all communities can benefit from. In this paper, we propose a framework that applies information extraction to generate abstract knowledge from source documents to be shared among participating communities. The framework also enables users to search for documents based on keywords or metadata as well as to search for extracted knowledge. This search is not restricted to the community the user is registered at but covers all registered communities and the data they are willing to share with others.

1 Introduction

The goal of the D-Grid Initiative¹ (German Grid Initiative) is to establish a grid infrastructure for education and research in Germany. Various communities participate in these efforts with different backgrounds, e.g., astrophysics, chemistry, humanities, and climate research. As of 2011, more than 20 projects and communities benefit from the computational power and storage provided by the grid infrastructure.

Currently, communities operate mostly independently from each other. However, many applications could benefit from access to information provided by different communities. Assume, for instance, one community hosts information about chemical substances, ideally in machine-readable semantic formats, but more likely in text files. Such knowledge might also be of use to other communities that need to look up details on a specific chemical substance mentioned in one of their resources. In such cases, communities can benefit from each other and build synergies by sharing their information. The big show-stopper for such effective inter-community collaborations so far has been finding and accessing relevant data efficiently across several communities. A user knows only her own community and how to search there, but has usually no idea how to search in other communities; neither does she know which search engine to use and how to formulate the query nor to which community to address her search request.

This paper introduces a framework for sharing information between grid communities that is being developed in the context of the WisNetGrid project². It enables users with

¹<http://www.d-grid.de/>

²<http://www.wisnetgrid.org/>

a minimal knowledge about other communities to find relevant information from any community. To achieve this goal, our framework provides a *search engine*, which allows users to formulate queries against a central instance and to specify a subset of communities that the search should consider. Transparently to the user, the query is rewritten, if necessary, and forwarded to registered search engines hosted by the communities specified by the user. These *local search engines* evaluate the query against their local data and report the result back to the central instance, where they are combined and presented to the user.

While standard document retrieval based on keyword search can satisfy many information needs, it is insufficient especially for complex and specific information needs. Often, users are not interested in a document containing a given set of words, but rather in some abstract information that may be spread over various documents. For instance, assume we search for all chemical substances that can reduce the amount of cesium being absorbed by fruits grown in Japan's Fukushima prefecture. By chance, there could be a document discussing exactly this kind of chemical substances, but in general it is likely that the information need can only be answered by combining information from multiple source documents. In order to answer such descriptive queries properly, the factual information contained in the available data needs to be extracted and converted into a machine readable semantic format, e.g. RDF³ or OWL⁴ that can be effectively queried. To achieve this goal, our framework provides an *information extraction* component that enables participating communities to extract semantic information contained in their data. By sharing the generated abstract ontologies, a *common knowledge space* is created. In contrast to keyword based search this also has the advantage that different ways to express semantically identical information in documents is unified into an abstract representation. Based on the extracted knowledge, users are able to formulate queries using structured query languages that cannot be expressed with keywords and answered by keyword search engines.

In this paper, we present a framework that allows communities to efficiently share information by applying information extraction and comprehensive search. In the following, we first highlight the architecture of the proposed framework (Section 2). In Section 3, we discuss details on how to work with the system and conclude the paper in Section 4.

2 Architecture

Before we go into details on how the framework enables sharing information between grid communities (Section 2.2), let us first review local search in a community (Section 2.1).

2.1 Local Search in a Community

Each grid community maintains community-specific data. Some communities store their data within the grid, others store their data on servers outside the grid infrastructure. In any case, the users of a community work with this data, obtain new data, analyze it and store the results. As a natural consequence of the amount of data, communities sooner or later need a search engine to find relevant data, e.g., for new experiments or to look up information. Thus, each community usually maintains a document search engine for its

³<http://www.w3.org/RDF/>

⁴<http://www.w3.org/TR/owl2-overview/>

data such as Lucene⁵ or TopX [TBM⁺08].

In general, the search functionality is hosted at a server outside the grid structure (*community server*, Figure 1 right-hand side). Of course, a community might use multiple search engines that might run on different servers. Access to data sources might be restricted to specific users or user groups (*access layer*).

On the other hand, the amount of data that needs to be analyzed, e.g., thousands of texts, often requires extracting information automatically from source documents [Sar08] and store it in a condensed machine-readable format for future use and reference, e.g., in the form of ontologies and RDF. Alternatively, ontologies are created manually by users and shared within the community. Systems for RDF data management, such as Sesame⁶, RDF-3X [NW10], RDF column stores [SGK⁺08] and Hexastore [WKB08], efficiently answer structured queries against the extracted information. These systems are also hosted at servers outside the grid infrastructure (*community server*) and available to all users of a community.

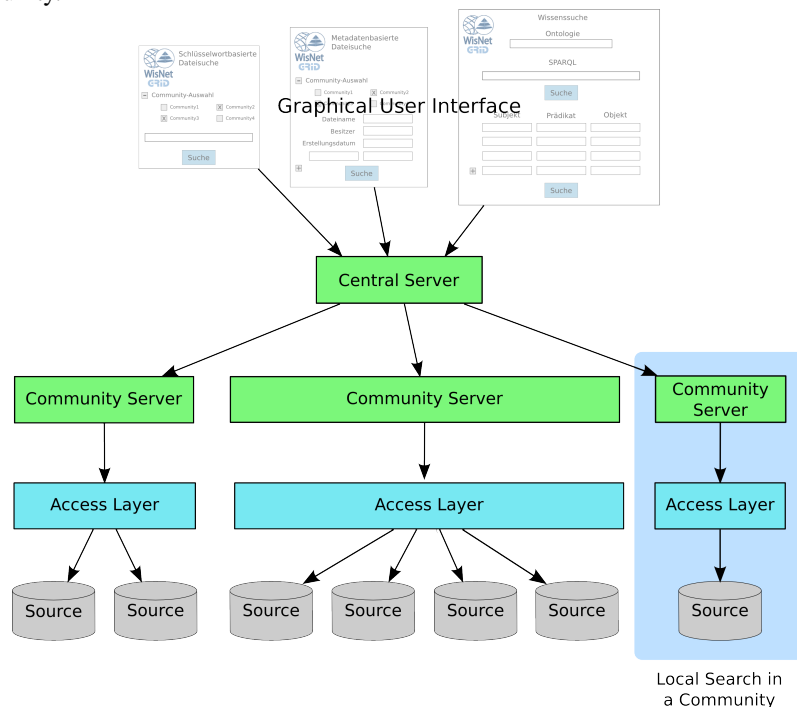


Figure 1: System Architecture

2.2 Comprehensive Search across Multiple Communities

The community search engines operate independently from each other. To query information provided by a community, potential users of other communities need to know the search engine and how to use it. Searching for information in multiple communities is therefore often a tedious task that the framework proposed in this paper tries to overcome.

⁵<http://lucene.apache.org/>

⁶<http://www.openrdf.org/>

The proposed framework (Figure 1) adds a federation layer on top of existing community search engines and services. A *central server* provides a graphical user interface for query formulation. A query at the central server is executed in a federated fashion by forwarding the query to registered *community search engines*, merging their results, and outputting them to the user. This works for keyword queries as well as for RDF queries.

By introducing a central server and a graphical user interface, all users can formulate queries and have their queries evaluated transparently on the data of all participating communities. Although the central server collaborates with the community search engines to answer a global query, extracting knowledge from documents (information extraction) and indexing are still tasks of the communities, which might decide to run these processes on the grid infrastructure. This guarantees that the communities are in full control over the data they wish to index and share with other communities.

Knowledge extraction on large corpora is a resource intensive task. Therefore, communities are encouraged to store their extraction results permanently in ontology files and share this RDF data with other communities. Beyond a broadened knowledge base available to human users, sharing ontologies can also enhance the extraction process of other communities, as information extraction can benefit from ontologies in two ways. First, existing knowledge of a domain is used to guide several extraction steps, e.g., determine which entity is referenced in a text by trying to find matches for related entities in the text. Second, the meta information about how factual knowledge of certain relations manifests itself in documents, i.e., *textual patterns*, can also be exported into an ontology and reused on similar documents. To enable users to find ontologies covering a relevant knowledge domain, the global server provides a specific search functionality.

3 Workflow

In this section we introduce the basic workflows of information extraction and federated search, i.e., how users perceive the system and interact with it.

3.1 Information Extraction

The information extraction approach we describe in the following is based on the SOFIE/PROSPERA system [SSW09,NTW11]. However, the framework can also integrate other IE systems.

The information extraction mechanism we employ extracts binary relations between uniquely identified entities using a pattern based approach. It requires background knowledge that depends on the extraction domain. First, a set of typed entities needs to be provided together with their possible textual representations. Second, for each relation (RDF predicate) to be considered, textual examples must be supplied. As an example, assume that we want to extract information about the half-life of radioactive elements from text documents. Here, we want to extract triples with a relation `hasHalfLife` (RDF predicate), a domain `RadioactiveElement` (RDF subject), and a range `Duration` (RDF object). The input needed for our extraction system is a list of chemical elements, typed as instances of `RadioactiveElement` along with their name variants and abbreviations. Furthermore, the system needs some instances of the relation `hasHalfLife` and some textual examples expressing these instances. From these, the system will learn textual patterns expressing the relation automatically [NTW11]. After having extracted

fact candidates, the system employs a reasoning step based on manually configurable logical rules to filter inconsistent information.

Given the background knowledge, the information extraction engine running on a community server can be started, observed, and controlled by the user. From a user's perspective the main extraction steps are:

1. The user provides background knowledge and the source documents to parse.
2. The extraction component parses the input documents and uses the provided background knowledge to disambiguate entities and identify candidates for the relations the user wishes to extract.
3. As an optional step, the user can inspect the extracted fact candidates. She might decide to correct entity disambiguations or to remove fact candidates.
4. To avoid inconsistencies in the result, the extraction component runs a reasoning step based on logical rules that matches the extracted relation against the background knowledge.
5. The user is presented the output from the reasoning step and can edit or remove these facts.
6. The final result of the extraction process is stored at a location defined by the user, e.g., a specific knowledge base.

3.2 Federated Search

After having stored the extracted information in a knowledge base, it can be shared with other communities and searched using the global search functionality of our framework. Our system provides three types of global search: (i) searching for RDF data using SPARQL queries, (ii) searching for documents based on meta data (author, file name, timestamp, etc), or (iii) searching for documents using keywords.

Our approach works in a federated fashion so that we make use of the community search engines to execute a global query. The community search engines need to be registered at the central component. The general steps are the same for all query types:

1. A user formulates a query using the graphical user interface provided by the global search engine.
2. This query is forwarded to registered search engines hosted by the communities – in consideration of the query type, i.e., registered keyword search engines receive keyword queries, registered RDF knowledge bases receive SPARQL queries, and registered metadata search engines receive metadata queries.
3. These search engines receive the query and execute it.
4. The results are collected at the central component and combined into a global query result, which is output to the user.

Some query types require additional steps. For keyword search we can introduce an additional query expansion step that creates different combinations of keywords that potentially meet the user's intention (synonyms, hypernyms, hyponyms, etc.), i.e., for the set of keywords defined in the user query, we obtain multiple sets of keywords and therefore multiple keyword queries. The results of these queries are merged into one result. As this requires expert and domain knowledge, query expansion is not implemented at the central component but directly at the community search engines.

As the communities are completely independent from each other, it is possible that each community uses a different metadata format to store additional information about documents, or resources respectively. Hence, when searching for documents based on metadata, it is necessary to define a global metadata schema for use by the central component. Users formulate global queries against this metadata schema. The central component maintains mappings from all community schemas to the global schema. When a user issues a query, it is rewritten based on the mappings and sent to the community search engines.

When querying RDF data, we can additionally consider reasoning, i.e., inferring logical consequences based on the given RDF data. This often leads to new results that are not available with standard RDF processing.

4 Conclusion

In this paper, we have presented a framework that enables federated search spanning multiple grid communities. It enables communities to share and find information despite differences in metadata formats. Furthermore, our framework proposes information extraction techniques to gather knowledge contained in large sets of documents. Communities can benefit from each other not only by sharing their extraction results but also their extraction meta-knowledge. Applying the subsidiarity principle, the framework assigns most administrative control and tasks to the community level, such that, for instance, every community has control over which information is shared on which abstraction layer. Thus, domain-specific knowledge available at the community hosting the data can be used when extracting information, while a central server ensures that all shared resources can be found globally. We are currently in the process of implementing the proposed framework.

So far, we have considered reasoning for each community in separate. An additional interesting aspect, which we plan to investigate in future work, is distributed reasoning; by exploiting semantic links between knowledge bases of different communities, we can derive answers to queries that none of the communities can provide when considering their data in separate.

References

- [NTW11] Ndapandula Nakashole, Martin Theobald, and Gerhard Weikum. Scalable knowledge harvesting with high precision and high recall. In *WSDM*, 2011.
- [NW10] Thomas Neumann and Gerhard Weikum. The RDF-3X engine for scalable management of RDF data. *The VLDB Journal*, 19:91–113, February 2010.
- [Sar08] Sunita Sarawagi. Information Extraction. *Found. Trends databases*, 1:261–377, 2008.
- [SGK⁺08] Lefteris Sidiropoulos, Romulo Goncalves, Martin Kersten, Niels Nes, and Stefan Manegold. Column-store support for RDF data management: not all swans are white. *Proc. VLDB Endow.*, 1:1553–1563, 2008.
- [SSW09] Fabian M. Suchanek, Mauro Sozio, and Gerhard Weikum. SOFIE: a self-organizing framework for information extraction. In *WWW*, 2009.
- [TBM⁺08] Martin Theobald, Holger Bast, Debapriyo Majumdar, Ralf Schenkel, and Gerhard Weikum. TopX: efficient and versatile top-k query processing for semistructured data. *The VLDB Journal*, 17:81–115, January 2008.
- [WKB08] Cathrin Weiss, Panagiotis Karras, and Abraham Bernstein. Hexastore: sextuple indexing for semantic web data management. *Proc. VLDB Endow.*, 1:1008–1019, 2008.